기출예상문제_답안

AI Expert · 모범 답안

1부. 인공지능 기초 (260406~260409 / 장병탁 교수님)

문제 1. CNN 출력 크기 및 파라미터 수 계산

정답

출력 크기 공식 : Output = ⌊(Input - Kernel) / Stride⌋ + 1

1. Conv2D(3×3, stride=1, padding=0) : (28 - 3)/1 + 1 = 26 × 26 (채널 32)
2. MaxPool2D(2×2, stride=2) : (26 - 2)/2 + 1 = 13 × 13 (채널 32)
3. Conv2D(3×3, stride=1, padding=0) : (13 - 3)/1 + 1 = 11 × 11 (채널 64)
4. MaxPool2D(2×2, stride=2) : (11 - 2)/2 + 1 = 5 × 5 (채널 64) (11/2 내림 = 5)
5. Flatten 차원 = 64 × 5 × 5 = 1600
6. FC 파라미터 수 = (입력 차원 × 출력 차원) + bias = 1600 × 10 + 10 = 16,010

[채점 포인트]

- 출력 크기 공식 ⌊(I - K)/S⌋ + 1 적용 ✔
- 각 레이어의 채널 수와 공간 크기 정확 ✔
- Flatten 차원 (채널 × H × W) ✔
- FC 파라미터 수 (weight + bias) ✔

문제 2. RNN의 한계와 LSTM 게이트 구조

정답

1. 기울기 소실(Vanishing Gradient) 문제
   - 시간 방향으로 체인 룰이 반복적으로 곱해지면서, 시퀀스가 길어질수록 초기 시점의 그래디언트가 0에 가깝게 수렴하여 앞쪽 입력의 정보가 학습에 반영되지 않기 때문이다.
   - (또는 정보 병목(Information Bottleneck) 문제 : 고정 크기 은닉 상태에 모든 과거 정보를 압축해야 하므로 초기 정보가 소실됨)

2. LSTM의 3개 게이트
   - Forget Gate (망각 게이트) : 이전 셀 상태(cell state)에서 어떤 정보를 버릴지 결정한다.
   - Input Gate (입력 게이트) : 현재 입력에서 어떤 새로운 정보를 셀 상태에 저장할지 결정한다.
   - Output Gate (출력 게이트) : 셀 상태로부터 어떤 정보를 은닉 상태(출력)로 내보낼지 결정한다.

3. LSTM은 셀 상태(cell state)를 별도로 두고 게이트로 정보의 흐름을 조절함으로써, 중요한 정보를 오랜 시점에 걸쳐 유지하고 불필요한 정보는 잊을 수 있다. 따라서 기본 RNN이 다루지 못하는 장기 의존성을 효과적으로 학습할 수 있다.

[채점 포인트]

> ● 기울기 소실(또는 정보 병목) 언급 ✔
> ● Forget / Input / Output 게이트 이름 정확히 ✔
> ● 각 게이트 기능 1줄 이상 설명 ✔
> ● 장기 의존성 해결 언급 ✔

문제 3. Scaled Dot-Product Attention 계산

정답

1. 내적 점수
   - $Q \cdot K_1 = 1 \cdot 1 + 0 \cdot 0 = 1$
   - $Q \cdot K_2 = 1 \cdot 0 + 0 \cdot 1 = 0$

2. 스케일된 점수 ($\sqrt{2} \approx 1.414$ 로 나눔)
   - $s_1 = 1 / \sqrt{2} \approx 0.707$
   - $s_2 = 0 / \sqrt{2} = 0.000$

3. Softmax 가중치
   - $e^{0.707} \approx 2.028$, $e^0 = 1$
   - 합 = 2.028 + 1 = 3.028
   - $\alpha_1 = 2.028 / 3.028 \approx 0.67$
   - $\alpha_2 = 1.000 / 3.028 \approx 0.33$

4. Attention 출력
   - output = 0.67 × [1, 2] + 0.33 × [3, 4]
   - = [0.67, 1.34] + [0.99, 1.32]
   - ≈ [1.66, 2.66]

5. $\sqrt{d_k}$ 로 나누는 이유 : 차원 d_k가 커질수록 내적 $Q \cdot K^T$의 분산이 커져 softmax 입력값이 매우 커지면, 출력 분포가 한쪽으로 지나치게 치우쳐(saturate) 기울기가 소실된다. 이를 방지하기 위해 스케일링하여 softmax의 분포를 안정화한다.

[채점 포인트]

> ● Q·K 내적 계산 정확 ✔
> ● $\sqrt{d_k}$ 스케일링 적용 ✔
> ● Softmax 가중치 합 = 1 만족, 계산 정확 ✔
> ● 최종 출력 벡터 (가중합) 정확 ✔
> ● 스케일링의 목적(분산·gradient saturation 방지) 언급 ✔

문제 4. BERT vs GPT 구조·학습 비교와 In-Context Learning

정답

1. 사용하는 구조
   - BERT : Transformer Encoder 만 사용
   - GPT : Transformer Decoder 만 사용
2. 사전학습 목표
   - BERT : Masked Language Modeling (MLM). 입력 토큰 중 일부를 [MASK]로 가리고, 그 자리의 원래 토큰을 예측하도록 학습한다. (+ 경우에 따라 Next Sentence Prediction)
   - GPT : Next Token Prediction (Causal Language Modeling). 이전 토큰들로부터 다음 토큰을 예측하도록 학습한다.
3. 문맥 파악 방향
   - BERT : Bidirectional (양방향). 좌우 문맥을 동시에 본다.
   - GPT : Unidirectional (일방향). 왼쪽(과거) 문맥만 본다. 미래 토큰은 마스킹된다.
4. 태스크별 적합성
   - (가) 빈칸 채우기 / 분류 / 개체명 인식 → BERT가 적합
   - 이유 : 이 태스크들은 대상 토큰의 좌우 문맥을 모두 활용해야 풍부한 의미 표현을 얻을 수 있고, BERT의 MLM / 양방향 인코더가 이에 최적화되어 있기 때문이다.
   - (나) 프롬프트 이어받아 긴 글 생성 → GPT가 적합
   - 이유 : 텍스트 생성은 왼쪽에서 오른쪽으로 다음 토큰을 순차적으로 예측하는 문제이며, GPT의 일방향 디코더가 Next Token Prediction으로 이 문제에 자연스럽게 정합하기 때문이다.
5. In-Context Learning (Few-shot Learning)
   - 사전학습된 대규모 언어모델이 별도의 파라미터 업데이트(Fine-tuning) 없이, 프롬프트에 주어진 몇 개의 예시(zero/one/few-shot)와 지시만으로 새로운 태스크를 수행할 수 있는 능력을 말한다. 이는 모델의 규모(파라미터·데이터·연산)가 커지면 자연스럽게 나타나는 창발적(emergent) 능력으로, Scaling Law와 함께 LLM의 핵심 특성이다.

[채점 포인트]

> ● Encoder vs Decoder 정확히 구분 ✔
> ● MLM / Next Token Prediction 목표 정확히 ✔
> ● Bidirectional / Unidirectional 정확히 ✔
> ● 태스크별 적합성과 이유 연결 ✔
> ● In-Context Learning 개념 + fine-tuning 없이 수행 가능 언급 ✔

문제 5. Scaled Dot-Product Attention 계산

정답

1. 내적 점수

```
Q·K1 = 1 x 1 + 0 x 0 = 1
Q·K2 = 1 x 0 + 0 x 1 = 0
```

2. scaled score

```
s1 = 1 / sqrt(2) = 0.707
s2 = 0 / sqrt(2) = 0
```

3. softmax attention weight

```
alpha1 = e^0.707 / (e^0.707 + e^0)
       = 2.03 / (2.03 + 1)
       = 0.67

alpha2 = e^0 / (e^0.707 + e^0)
       = 1 / (2.03 + 1)
       = 0.33
```

4. 최종 Attention 출력 벡터

```
output = 0.67 x [2,0] + 0.33 x [0,2]
       = [1.34, 0] + [0, 0.66]
       = [1.34, 0.66]
```

5. sqrt(d_k)로 나누는 이유

차원이 커지면 QK^T 내적값의 크기가 커져 softmax가 한쪽으로 치우칠 수 있다. 이를 완화하고 attention 분포와 gradient를 안정화하기 위해 sqrt(d_k)로 나눈다.

[채점 포인트]

- Q와 K의 내적 계산
- sqrt(d_k) scaling 적용
- softmax 확률 계산
- V의 weighted sum 계산
- scaling 이유 설명

문제 6. BERT와 GPT 비교

정답

1. 사용하는 Transformer 구조

- BERT: Transformer Encoder-only
- GPT: Transformer Decoder-only

2. 사전학습 목표

- BERT: Masked Language Modeling(MLM). 문장 중 일부 토큰을 가리고 원래 토큰을 예측한다.
- GPT: Next Token Prediction. 앞의 토큰들을 보고 다음 토큰을 예측한다.

3. 문맥 처리 방향

- BERT: Bidirectional. 앞뒤 문맥을 모두 본다.
- GPT: Unidirectional/Causal. 앞 문맥만 보고 미래 토큰은 보지 않는다.

4. 태스크별 적합성

- 문장 분류 또는 개체명 인식: BERT

BERT는 앞뒤 문맥을 모두 활용해 문장 전체의 의미 표현을 만들기 좋으므로 이해 중심 태스크에 적합하다.

- 주어진 문장을 이어서 글 생성: GPT

GPT는 이전 토큰을 바탕으로 다음 토큰을 순차적으로 생성하도록 학습되었으므로 생성 태스크에 적합하다.

5. In-Context Learning

모델 파라미터를 fine-tuning하지 않고, 프롬프트 안에 지시문이나 몇 개의 예시를 함께 넣어 새로운 문제를 풀게 하는 방식이다. 예시가 없으면 zero-shot, 하나면 one-shot, 몇 개면 few-shot이라고 부른다.

[채점 포인트]

- Encoder-only / Decoder-only 구분
- MLM / Next Token Prediction 구분
- Bidirectional / Unidirectional 구분
- 이해 태스크와 생성 태스크의 적합성 설명
- fine-tuning 없이 프롬프트 예시로 수행한다는 설명

문제 7. CNN 출력 크기와 Flatten 차원 계산

정답

CNN 출력 크기 공식은 다음과 같다.

> H_out = floor((H_in - K + 2P) / S) + 1

1. 첫 번째 Conv2D

> H_out = (28 - 5 + 2 x 0) / 1 + 1 = 24

출력 크기: 16 x 24 x 24

2. 첫 번째 MaxPool2D

> H_out = (24 - 2) / 2 + 1 = 12

출력 크기: 16 x 12 x 12

3. 두 번째 Conv2D

> H_out = (12 - 3 + 2 x 1) / 1 + 1 = 12

출력 크기: 32 x 12 x 12

4. 두 번째 MaxPool2D

> H_out = (12 - 2) / 2 + 1 = 6

출력 크기: 32 x 6 x 6

5. Flatten 차원

> 32 x 6 x 6 = 1152

따라서 Flatten 후 벡터 차원은 1152이다.

[채점 포인트]

> - 출력 크기 공식 적용
> - kernel, stride, padding 의미 이해
> - pooling 후 크기 계산
> - Flatten = 채널 x 높이 x 너비

문제 8. RNN의 한계와 LSTM 게이트

정답

1. RNN은 현재 입력 x_t와 이전 hidden state h_{t-1}를 함께 사용하여 현재 hidden state h_t를 계산한다. 따라서 이전 시점의 정보가 hidden state를 통해 다음 시점으로 전달된다.

> $h_t = \tanh(W_h h_{t-1} + W_x x_t + b)$

2. 대표적인 문제는 기울기 소실(Vanishing Gradient) 이다.

긴 sequence에서는 시간 방향으로 chain rule이 반복 적용된다. 이때 작은 미분값이 계속 곱해지면 앞쪽 시점까지 전달되는 gradient가 거의 0이 되어 초기 입력 정보가 학습에 잘 반영되지 않는다.

3. LSTM의 3가지 gate

> - Forget Gate: 이전 cell state에서 어떤 정보를 잊을지 결정한다.
> - Input Gate: 현재 입력에서 어떤 정보를 새롭게 저장할지 결정한다.
> - Output Gate: cell state의 정보 중 어떤 것을 hidden state로 출력할지 결정한다.

4. LSTM은 hidden state와 별도로 cell state를 두고 gate로 정보의 삭제, 저장, 출력을 조절한다. 그래서 중요한 정보를 더 오래 유지할 수 있고, 기본 RNN보다 장기 의존성 문제를 완화할 수 있다. GRU는 LSTM을 더 단순화한 gate 기반 모델로, RNN보다 gradient 흐름을 더 잘 유지하도록 설계되었다.

[채점 포인트]

> - hidden state가 과거 정보를 전달한다는 설명
> - vanishing gradient와 chain rule 반복 언급
> - Forget/Input/Output gate 이름과 역할
> - cell state 또는 gate를 통한 장기 정보 유지 설명

2부. 딥러닝 기초 (260410~260421 / 윤성로 교수님)

문제 9. [생성형 AI 기초]

정답

판별형 모델은 조건부 확률 $P(y|x)$를 학습하여 입력 x를 하나의 정답 y로 분류·예측하는 반면, 생성형 모델은 데이터 분포 $P(x)$ 또는 결합분포 $P(x, y)$를 학습하여 그 분포에서 새로운 샘플을 만들어낼 수 있다. 즉 판별형은 결정 경계를 배우고, 생성형은 데이터의 분포 자체를 배워 샘플링이 가능하다는 점이 핵심 차이이다.

문제 10. [VAE (Variational Autoencoder)]

정답

일반 AE의 인코더는 입력 x를 단일 잠재 벡터 z로 결정론적(deterministic) 매핑하여 압축·복원에

집중하는 반면, VAE의 인코더는 잠재 분포 $q_\varphi(z|x)$의 파라미터(μ, σ^2)를 출력하고 그 분포에서 z를 샘플링한다. 이로 인해 VAE는 학습 후 디코더만 사용하여 prior에서 뽑은 새로운 z로 다양한 샘플을 생성할 수 있는 확률적 생성 모델로 동작한다.

문제 11. [VAE (Variational Autoencoder)]

정답

인코더가 내놓은 분포에서 직접 z를 샘플링하면 샘플링 연산이 미분 불가능하여 역전파로 인코더 파라미터를 학습할 수 없다. 이를 해결하기 위해 $z = \mu + \sigma \odot \varepsilon$ ($\varepsilon \sim N(0, I)$) 형태로 무작위성을 외부 입력 노이즈 ε로 분리하여, μ와 σ를 거쳐 그래디언트가 정상적으로 흐르도록 만드는 기법이다.

문제 12. [VAE (Variational Autoencoder)]

정답

ELBO는 Reconstruction 항 $E_{z\sim q_\varphi(z|x)}[\log p_\theta(x|z)]$ 과 KL 항 $-D_{KL}(q_\varphi(z|x) \,/\!/\, p(z))$ 로 구성된다. Reconstruction 항은 샘플링된 z를 디코더에 통과시켰을 때 원본 x가 얼마나 잘 복원되는지를 평가하는 훈련 손실 역할을 하고, KL 항은 인코더 분포를 prior(일반적으로 $N(0, I)$)에 가깝게 끌어당기는 정규화(regularizer) 역할을 하여 잠재 공간이 연속적이고 부드럽게 채워지게 한다.

문제 13. [GAN (Generative Adversarial Network)]

정답

Generator는 무작위 노이즈 z를 입력받아 진짜처럼 보이는 가짜 샘플 G(z)를 만들어 Discriminator를 속이는 것을 목표로 하고, Discriminator는 입력이 진짜 데이터인지 Generator가 만든 가짜인지를 0~1 확률로 구분하도록 학습한다. 두 네트워크는 Minimax 목적함수 하에서 적대적(adversarial)으로 경쟁하며 함께 성능이 향상된다.

문제 14. [GAN (Generative Adversarial Network)]

정답

학습 초반에는 Generator가 만든 샘플이 조잡하여 Discriminator가 쉽게 구분해 $D(G(z)) \approx 0$ 이 되는데, 이 영역에서 $\log(1 - D(G(z)))$의 기울기가 매우 작아(포화되어) Generator에 충분한 학습 신호가 전달되지 않는다. $\log D(G(z))$를 최대화하는 형태로 목적을 뒤집으면 같은 영역에서 기울기가 크게 유지되어 학습 초반에도 Generator가 안정적으로 업데이트될 수 있기 때문이다.

문제 15. [GAN (Generative Adversarial Network)]

정답

Generator가 데이터 분포의 다양한 모드(유형) 중 일부만 반복적으로 생성하여 다양성(diversity)이 사라지는 현상이다. 예를 들어 MNIST 학습 시 0~9 중 특정 숫자만 계속 만들어내는 경우로, Minimax 학습의 불안정성과 Generator가 Discriminator를 속이기 쉬운 모드로 수렴해 버리는 데서 발생한다.

문제 16. [생성 모델 평가]

정답

FID는 사전학습된 Inception v3 네트워크의 중간 feature 공간에서 진짜 이미지 분포와 가짜 이미지

분포를 각각 다변량 Gaussian으로 근사 한 뒤, 두 분포의 평균 μ와 공분산 Σ 간 Fréchet distance(Wasserstein-2 distance)를 계산한 값이다. 값이 낮을수록 두 분포의 통계량이 가까워 생성 이미지가 진짜 이미지와 feature 수준에서 구분되지 않음을 의미하므로 생성 품질이 좋다고 판단한다.

문제 17. Autoencoder와 VAE의 차이

정답

Autoencoder는 입력 x를 encoder로 압축해 latent representation h를 만들고, decoder로 다시 x를 복원하도록 학습한다. 하지만 일반 Autoencoder의 latent space는 우리가 아는 단순한 분포로 정리되어 있지 않아, 임의로 샘플링해서 새로운 데이터를 생성하기 어렵다.
VAE는 encoder가 하나의 벡터가 아니라 latent distribution의 파라미터, 예를 들어 mu와 sigma를 출력하게 한다. 그리고 KL divergence regularization을 통해 latent distribution이 표준정규분포 같은 prior에서 크게 벗어나지 않도록 묶는다. 그래서 학습 후에는 prior에서 z를 샘플링해 decoder에 넣어 새로운 데이터를 생성할 수 있다.

[채점 포인트]

- Autoencoder = encoding/decoding으로 reconstruction
- VAE = latent distribution을 학습
- prior distribution에 맞추는 KL regularization 언급
- VAE는 sampling을 통한 generation 가능

문제 18. VAE Loss의 두 구성 요소

정답

Reconstruction Loss는 encoder와 decoder를 거친 출력이 원래 입력과 얼마나 비슷한지를 측정한다. 즉 입력을 잘 복원하도록 decoder를 학습시키는 항이다. 연속 데이터에서는 MSE, 이진 데이터에서는 Binary Cross Entropy, categorical 데이터에서는 Cross Entropy를 사용할 수 있다.
KL Divergence 항은 encoder가 만든 latent distribution q(z|x)가 우리가 정한 prior distribution p(z), 보통 표준정규분포에서 너무 멀어지지 않도록 제한하는 regularizer 역할을 한다. 이 항 덕분에 latent space가 샘플링 가능한 형태로 정리되고, VAE가 생성 모델로 동작할 수 있다.

[채점 포인트]

- Reconstruction Loss = 입력 복원 품질
- KL Divergence = q(z|x)를 p(z)에 가깝게 함
- KL 항이 regularizer 역할을 한다는 설명
- 데이터 타입에 따라 reconstruction loss가 달라질 수 있음

문제 19. Reparameterization Trick

정답

VAE는 encoder가 출력한 분포에서 latent variable z를 샘플링한 뒤 decoder에 넣는다. 그런데

단순한 random sampling은 미분 가능하지 않아 backpropagation으로 encoder까지 gradient를 전달하기 어렵다.
Reparameterization Trick은 z를 직접 샘플링하지 않고, epsilon ~ N(0, I)를 샘플링한 뒤 z = mu + sigma * epsilon처럼 표현한다. 이렇게 하면 randomness는 epsilon으로 분리되고, mu와 sigma에 대해서는 미분 가능해져 VAE 전체를 gradient descent로 학습할 수 있다.

[채점 포인트]

- sampling은 미분 불가능하다는 문제
- z = mu + sigma * epsilon 형태
- randomness와 학습 파라미터 분리
- backpropagation 가능하게 함

문제 20. GAN의 Generator와 Discriminator

정답

Generator는 단순한 noise vector z를 입력받아 실제 데이터처럼 보이는 fake sample을 생성하는 네트워크이다. 목표는 Discriminator를 속일 만큼 그럴듯한 데이터를 만드는 것이다.
Discriminator는 입력 sample이 real data에서 온 것인지, Generator가 만든 fake data인지 구분하는 네트워크이다. 목표는 real sample은 real로, fake sample은 fake로 정확히 분류하는 것이다.
GAN은 이 두 네트워크가 서로 경쟁하는 two-player minimax game으로 학습된다. Discriminator가 좋아질수록 Generator는 더 진짜 같은 sample을 만들도록 학습된다.

[채점 포인트]

- Generator = noise z에서 fake sample 생성
- Discriminator = real/fake 판별
- 두 모델이 적대적으로 학습
- minimax game 또는 game theory 언급

문제 21. GAN의 Minimax Objective와 Mode Collapse

정답

GAN은 Generator와 Discriminator가 서로 반대 목적을 가지고 번갈아 학습하는 구조이다.
Discriminator는 진짜와 가짜를 잘 구분하려 하고, Generator는 Discriminator를 속이려 한다. 두 네트워크의 균형이 맞지 않으면 학습이 발산하거나 한쪽이 너무 강해져 gradient가 제대로 전달되지 않을 수 있다.
Mode Collapse는 Generator가 다양한 sample을 만들지 못하고 비슷한 sample만 반복해서 생성하는 현상이다. 예를 들어 여러 종류의 이미지를 만들어야 하는데 특정 클래스나 특정 형태의 이미지만 계속 만드는 경우이다. GAN은 sharp한 sample을 잘 만들 수 있지만, 초기 GAN에서는 diversity가 떨어지는 문제가 자주 발생했다.

[채점 포인트]

- Generator와 Discriminator의 경쟁 구조
- 학습 균형이 깨지면 unstable
- Mode Collapse = 비슷한 sample만 반복 생성
- quality-diversity trade-off 언급 가능

문제 22. VAE와 GAN의 장단점 비교

정답

VAE는 latent distribution을 prior에 맞추며 학습하므로 latent space가 비교적 안정적이고 다양한 sample을 생성하기 쉽다. 하지만 초기 VAE는 reconstruction 중심 학습 때문에 생성 이미지가 흐릿해지는(blurry) 문제가 있었다.
GAN은 Discriminator와 경쟁하면서 학습하므로 상대적으로 선명하고 현실적인 sample을 만들 수 있다. 하지만 학습이 불안정하고, Generator가 특정 mode에만 몰리는 Mode Collapse로 인해 sample diversity가 떨어질 수 있다.
요약하면 VAE는 diversity와 안정성 쪽에 강점이 있고, GAN은 sample quality 쪽에 강점이 있지만 training stability와 diversity 문제가 있다.

[채점 포인트]

- VAE = 안정적, 다양한 sample, blurry 가능
- GAN = sharp/high-quality sample
- GAN = unstable training, mode collapse 가능
- quality-diversity trade-off 설명

문제 23. Batch Normalization과 Dropout

정답

Batch Normalization은 mini-batch의 feature 통계를 이용해 activation을 정규화한 뒤 learnable scale과 bias로 다시 조정하는 기법이다. 학습을 안정화하고 gradient flow를 개선하며, 보통 convolution layer와 ReLU 사이에 들어간다.
Dropout은 학습 중 일부 neuron을 random하게 꺼서 모델이 특정 neuron 조합에 과도하게 의존하지 않도록 하는 regularization 기법이다. overfitting을 줄이고 일반화 성능을 높이는 데 도움이 된다.
Training mode에서는 BatchNorm이 현재 batch 통계를 사용하고 Dropout이 random하게 neuron을 끈다. Evaluation mode에서는 BatchNorm이 학습 중 누적한 running mean/variance를 사용하고, Dropout은 꺼져 deterministic하게 동작한다.

[채점 포인트]

- BatchNorm = feature normalization + learnable scale/bias
- Dropout = neuron random masking으로 overfitting 완화
- train mode와 eval mode 차이
- eval에서는 deterministic해야 함

문제 24. Residual Connection의 필요성

정답

Residual Connection은 layer block이 직접 H(x)를 학습하는 대신, 입력 x에 더할 변화량인 residual F(x)=H(x)-x를 학습하도록 만든다. 출력은 보통 H(x)=F(x)+x 형태가 된다.
이렇게 하면 block이 입력 feature를 완전히 새로 변환하기보다 기존 feature를 유지하면서 필요한 만큼만 refine할 수 있다. 또한 gradient가 shortcut 경로를 통해 더 잘 흐르므로 매우 깊은 네트워크에서도 vanishing gradient 문제를 완화하고 안정적으로 학습할 수 있다.
단, F(x)와 x를 더하려면 두 tensor의 shape이 맞아야 하므로, convolution block에서는 padding 등을 사용해 height/width와 channel 차원을 맞추는 것이 중요하다.

[채점 포인트]

- H(x)=F(x)+x 구조
- residual을 학습하는 것이 더 쉬움
- 깊은 네트워크에서 gradient flow 개선
- 더하기 위해 shape matching 필요

3부. 자연어처리 (260427~260504 / 황승원 교수님)

문제 25. [Chain-of-Thought (CoT) — 강사 직접 지목]

정답

(정답 누락)

문제 26. [Test-Time Scaling — Self-Consistency / PoT — 강사 직접 지목]

정답

(정답 누락)

문제 27. [BERT의 사전학습 목표 — 미커버 영역]

정답

(정답 누락)

문제 28. [Self-Refinement (Self-Reflection) — Test-Time Scaling Sequential]

정답

(정답 누락)

문제 29. [Chain-of-Thought (CoT)]

정답

Standard prompting은 모델이 단번에 정답 토큰을 생성해야 하므로, 한 번의 forward 계산만으로는 정보 밀도가 낮은 토큰들이 답을 결정짓게 된다. CoT는 모델이 중간 추론 단계를 토큰으로 풀어내도록 유도하여 더 많은 토큰을 소비해 확률 분포를 sharper 하게 만든다. 즉 어려운 reasoning일수록 추가적인 계산(추가 토큰)이 필요하다는 원리에 기반하며, 그 결과 GSM8K 같은 reasoning 벤치마크에서 큰 폭의 성능 향상이 발생한다.

(3)

CoT는 small model에서는 거의 효과가 없고, 일정 사이즈 이상의 large language model에서만 성능 갭이 갑자기 벌어지는 Emergent Ability 이다. 이는 추론 과정 자체를 자연어로 매끄럽게 생성할 수 있는 언어 능력 + reasoning 패턴 인식이 큰 모델에서야 동시에 안정적으로 발현되기 때문이며, 작은 모델은 rationale을 생성하더라도 일관된 reasoning이 깨져 오히려 성능이 떨어질 수 있다.

문제 30. [Test-Time Scaling — Self-Consistency / PoT]

정답

Self-Consistency 는 동일한 prompt에 대해 CoT를 여러 번 (N번) 생성한 뒤, 각 reasoning chain이 도출한 최종 답들 중 가장 많이 나온 답을 다수결(majority voting)로 선택 하는 기법이다. CoT는 추론 경로를 1개만 만들지만 Self-Consistency는 여러 개를 만들어 답의 안정성을 높이므로, CoT의 확장형(CoT-SC) 으로 결합되어 사용된다.

(2)

Self-Consistency 는 여러 응답을 동시에 생성한 뒤 선택하는 구조이므로 Parallel 카테고리에 해당한다. Sequential 쪽 대표 기법으로는 Thinking Mode (Long CoT, o1/o3/DeepSeek-R1 류) 또는 Self-Refinement 가 있다.
(채점 포인트: Parallel 명시 + Sequential 예시 1개)

(3)

Program-of-Thought (PoT) 는 자연어 rationale 대신 모델이 Python 같은 코드를 생성하고, 그 코드를 외부 인터프리터로 실행시켜 답을 얻는다는 점에서 일반 CoT와 다르다. LLM이 산술에 약한 이유는 토큰화 방식 때문 인데, "1723" 같은 숫자가 사람에게는 통째로 한 숫자이지만 토크나이저는 "17" + "23" 처럼 쪼개고 띄어쓰기에 따라 같은 숫자가 다른 토큰이 되는 등, 인간이 인식하는 숫자와 모델이 인식하는 토큰이 일치하지 않기 때문이다. PoT는 이 문제를 외부 컴퓨테이션으로 우회한다.

문제 31. [BERT의 사전학습 목표]

정답

- MLM (Masked Language Modeling): 입력 문장에서 일부 토큰(약 15%)을 [MASK]로 가리고, 모델이 가려진 자리의 원래 토큰이 무엇인지 예측하도록 학습하는 방법. 양쪽 문맥을 모두 보고 빈칸을 채우는 방식.

- NSP (Next Sentence Prediction): 두 문장 A, B를 입력으로 주고, B가 실제로 A의 다음 문장인지 (IsNext) 아니면 무관한 문장인지 (NotNext)를 이진 분류로 예측. 문장 간 관계를 학습시키기 위한 보조 task.

(2)
GPT는 left-to-right autoregressive LM 이므로 t번째 토큰을 예측할 때 t보다 이전의 토큰만 볼 수 있는 단방향 모델이다. 반면 BERT는 MLM으로 빈칸을 맞추는 task이기 때문에, 빈칸의 왼쪽 context와 오른쪽 context를 동시에 인코더가 참조할 수 있어 bidirectional 표현 을 학습한다. 이로 인해 NLU(텍스트 이해) 계열 task에서 GPT보다 강한 표현력을 가지게 된다.

문제 32. [Self-Refinement / Self-Reflection]

정답

Self-Refinement는 한 번의 generation으로 답을 확정하지 않고 다음 흐름을 거친다.
1. Generate: 모델이 입력 prompt에 대해 초기 응답(Initial Output)을 생성.
2. Critique: 동일 모델이 자신의 출력을 입력으로 받아 잘못된 점·부족한 점을 평가하는 feedback을 생성.
3. Refine: 그 feedback을 다시 입력으로 받아 출력을 수정한 Refined Output을 생성.
만족스러울 때까지 (또는 정해진 횟수만큼) 이 사이클을 반복한다.

(2)
같은 모델이라도 task별로 발현되는 능력치가 다르기 때문이다. 즉 Initial Generation Task (정답을 처음 생성), Critique Task (틀린 곳 찾기), Refinement Task (피드백을 받고 수정) 은 각각 별개의 sub-skill 이며, 모델은 자신이 한 번에 풀기 어려운 문제도 "자신의 출력을 비판적으로 평가" 하는 과제로 바꾸면 더 잘 풀 수 있다. 이 원리 때문에 Claude Code, Cursor 같은 에이전트가 에러 메시지를 받아 스스로 수정하는 동작이 가능하며, "Multi-Agent" 도 결국 같은 모델에 다른 system prompt를 부여하여 역할을 분리한 self-loop의 확장이다.

문제 33. Language Model과 Autoregressive Generation

정답

(1) 언어 모델은 주어진 맥락(context) 다음에 어떤 단어/token이 등장할 확률을 모델링하는 모델이다. 즉 $P(w_t \mid w_1, \dots, w_{t-1})$를 예측한다.

(2)

$$
P(w_1, w_2, \dots, w_T)
= \prod_{t=1}^{T} P(w_t \mid w_1, \dots, w_{t-1})
$$

(3) Autoregressive generation은 지금까지 생성한 token들을 다시 context로 넣고 다음 token을 하나씩 순차적으로 생성하는 방식이다. GPT가 decoder-only causal language model이므로 이에 더

직접적으로 해당한다. BERT는 입력 전체를 보고 masked token을 예측하는 encoder-only 모델이므로 일반적인 autoregressive generation 모델은 아니다.

문제 34. Pre-training, Instruction Tuning, RLHF의 차이

정답

(1) Pre-training은 대량의 일반 텍스트로 next-token prediction을 학습하는 단계이다. 언어 패턴, 문법, 지식, 상식을 배운다.

(2) Instruction Tuning/SFT는 instruction-output 쌍을 이용해 "이런 명령이 오면 이런 답변을 해야 한다"는 행동을 학습시키는 단계이다. 모델을 instruction-following assistant에 가깝게 만든다.

(3) RLHF는 사람의 선호 데이터를 이용해 reward model을 학습하고, PPO 같은 강화학습으로 사람이 더 선호할 답변을 생성하도록 조정하는 단계이다.

Pre-training만 한 모델은 기본적으로 "사용자 요청에 답하는 모델"이 아니라 "주어진 텍스트 뒤에 올 법한 텍스트를 이어 쓰는 모델"이다. 그래서 질문에 답하는 대신 비슷한 질문 패턴이나 뒤따를 법한 문장을 계속 생성할 수 있다.

문제 35. T5의 Text-to-Text Framework와 Prefix

정답

(1) T5는 C) Encoder-Decoder 모델이다.

(2) Text-to-text framework는 번역, 요약뿐 아니라 분류나 회귀 문제까지 모두 "텍스트 입력 → 텍스트 출력" 문제로 바꾸는 방식이다. 예를 들어 문법성 판단도 class 확률을 직접 출력하는 대신 acceptable 또는 not acceptable이라는 문자열을 출력하게 만들 수 있다.

(3) Prefix는 입력이 어떤 task인지 모델에게 알려주는 표시이다. 예를 들어 translate English to German: ..., summarize: ..., cola sentence: ...처럼 prefix를 붙이면 같은 encoder-decoder 모델과 같은 decoder를 사용하더라도 task를 구분할 수 있다.

문제 36. T5 Span Corruption과 일반 Language Modeling 비교

정답

(1) Span corruption은 입력 문장에서 하나의 token이 아니라 연속된 여러 token span을 가리고, decoder가 그 가려진 span을 생성하도록 학습하는 방식이다.

(2) 일반 autoregressive language modeling은 앞쪽 context를 보고 뒤쪽 token을 순서대로 예측한다. 반면 span corruption은 문장의 앞, 중간, 뒤 어디든 일부 span을 가릴 수 있고, 모델은 손상된 입력 전체를 이해해서 빠진 span을 복원해야 한다.

(3) Span corruption은 encoder가 입력 전체의 정보를 더 고르게 이해하도록 만든다. 단순히 바로 앞 context만 잘 보면 되는 next-token prediction보다 어려운 denoising task이므로, encoder-decoder 모델이 입력 전체를 압축하고 활용하는 능력을 기르는 데 도움이 된다.